

Pesquisa técnica aprofundada

Pesquisa técnica aprofundada — Bootstrap com clone3 em um Docker-like system from scratch

Arquitetura Python + Cython + C++ para criação controlada do processo filho, namespaces, `exit_signal` e raw syscall wrapper

Documento orientado ao desenho de um runtime de containers “Docker-like” com control plane em Python, cola em Cython e data plane em C++.

Data de geração: 12 de março de 2026.

Escopo: feature de bootstrap com `clone3()`, namespaces, `exit_signal` e raw syscall wrapper.

Sumário

Resumo executivo

Escopo e premissas

1. O problema arquitetural e o contexto do Docker original
2. clone3 como superfície central do bootstrap
3. O que o runc ensina sobre a realidade do bootstrap
4. Proposta de arquitetura para o projeto Python + Cython + C++
5. System Design do subsistema de bootstrap
6. Low Level Design do bootstrap
- 6.1. Sketch de implementação para orientar a construção
7. Domain-Driven Design aplicado ao bootstrap
8. Padrões de Design GoF adequados a esta feature
9. Requirements funcionais e não funcionais
10. Plano incremental de implementação
11. Estratégia de testes e análise de falhas
12. Conclusão

Referências

Resumo executivo

O núcleo desta feature não é “criar um processo” e sim materializar, de forma determinística, a fronteira de isolamento do container. No Docker original, o daemon conversa com APIs e gerencia objetos, mas a execução concreta do ciclo de vida é delegada a camadas especializadas; hoje o Docker Engine opera como aplicação cliente-servidor, usa `dockerd` para expor API e delega o ciclo de vida dos containers ao `containerd`, que por padrão usa o `runc` como runtime. O próprio projeto Moby se apresenta como um conjunto modular de peças para montar sistemas de containers, não como uma aplicação monolítica. Essa separação de responsabilidades é a referência correta para o seu desenho: Python deve representar política, metadados, API e orquestração; C++ deve representar mecanismo, isolamento, syscalls e supervisão; Cython deve funcionar como fronteira tipada e não como local de lógica de kernel pesada. ^[1]
^[2]^[3]^[4]

O `clone3()`, introduzido no Linux 5.3, é especialmente adequado para esse papel porque ele organiza a criação do filho em torno de uma estrutura extensível, com campos explícitos para `flags`, `pidfd`, `exit_signal`, pilha, seleção de PIDs e cgroup de destino. Isso reduz ambiguidade, melhora a validabilidade da chamada e cria espaço para evoluções sem remodelar toda a API nativa. Em um runtime de containers, no entanto, o uso correto de `clone3()` exige mais do que um wrapper: é necessário um plano de bootstrap que saiba quando criar namespaces novos, quando entrar em namespaces já existentes, quando pedir mapeamentos de `uid/gid`, quando separar `create` de `start`, quando supervisionar por `pidfd` e quando fazer `execve()` do processo final. ^[5]
^[6]^[8]

A principal recomendação arquitetural deste relatório é não acoplar o bootstrap nativo ao processo Python que atende API. A documentação oficial do Python é explícita ao dizer que chamadas de `fork()` ou similares que clonem o processo atual só deveriam ocorrer a partir da thread principal do interpretador principal; além disso, o ecossistema Python passou a expor de forma mais agressiva os problemas de compatibilidade entre `fork` e processos multithread, a ponto de o método `fork` ter deixado de ser o padrão do `multiprocessing` em POSIX no Python 3.14. Em termos práticos, o caminho seguro é fazer o Cython chamar um runtime worker nativo, single-threaded, escrito em C++, e deixar esse worker executar a sequência `clone3 → sync → setns/unshare/mappings → exec`. ^[27]^[28]
^[29]

No desenho proposto, o runtime nativo deve tratar `pidfd` como identificador canônico do processo, usar `exit_signal = SIGCHLD` no MVP, validar combinações ilegais de flags antes de entrar no kernel, separar claramente

`create` de `start` em estilo OCI e modelar namespaces como intenção de domínio, não como um inteiro bruto de bits. A implementação deve assumir que o bootstrap é um subsistema próprio, com ciclo de vida, eventos, supervisão, limpeza e testes dedicados. A meta não é apenas funcionar em um caso feliz, mas permanecer correta diante de rootless mode, user namespaces, join de namespaces, falhas parciais, morte precoce do filho, compatibilidade de kernel e observabilidade operacional. [5][6][8][18][19][23][24][25]

Escopo e premissas

Este documento foca especificamente a feature de bootstrap baseada em `clone3()`, com ênfase em criação controlada do processo filho, flags de namespace, `exit_signal` e raw syscall wrapper, sempre no contexto de um runtime de containers Linux inspirado em Docker/OCI. Assuntos como build de imagens, distribuição, snapshotters, overlayfs, seccomp, capabilities, drivers de rede e armazenamento aparecem apenas quando interferem diretamente no bootstrap. A premissa é que o host é Linux, que a camada de metadados e API vive em Python e que a execução sensível ao kernel vive em C++, com Cython como fronteira de integração. [1][2][5][6][8][21][22]

1. O problema arquitetural e o contexto do Docker original

Construir um sistema “Docker-like” do zero costuma parecer, à primeira vista, um exercício de montar algumas syscalls e executar um processo em novos namespaces. Essa leitura é insuficiente. O Docker Engine não se resume ao runtime: ele se apresenta como uma aplicação cliente-servidor, com daemon de longa duração, APIs e CLI, e o próprio daemon gerencia objetos como imagens, containers, redes e volumes. Ao mesmo tempo, o Docker moderno não trata o runtime como detalhe interno invisível; ele usa o `containerd` para o ciclo de vida dos containers, e este, por padrão, usa o `runc` como runtime OCI. O `containerd`, por sua vez, se descreve como runtime voltado a simplicidade, robustez e portabilidade, desenhado para ser embutido em sistemas maiores e responsável por imagem, execução, supervisão e anexos de storage e rede. [1][2][3]

Esse arranjo é relevante porque o seu projeto quer replicar a mesma separação conceitual, mas com outra pilha de linguagem: Python no control plane, C++ no data plane e Cython como cola. A escolha é tecnicamente sólida desde que cada camada permaneça no seu lugar. Python deve carregar o modelo de domínio, os metadados persistidos, a API pública, a política de lifecycle e a compatibilização com um contrato “OCI-like”. C++ deve portar tudo que depende de invariantes de kernel, ABI, manipulação de file descriptors, sincronização entre processos,

sinalização e acesso às interfaces do hardware e do kernel. O Cython deve ser uma camada de tradução explícita entre intenção e mecanismo, e não um segundo domínio accidental. [4][5][6][21][22]

O recorte correto para esta feature, portanto, é o bootstrap do container. Pelo ponto de vista da especificação OCI, o runtime tem um ciclo de vida em que a fase de `create` materializa o ambiente do container sem executar ainda o programa do usuário, e a fase de `start` executa o processo configurado. O estado de um container transita por `creating`, `created`, `running` e `stopped`, e em Linux o estado expõe o `pid` do processo do container. Esse detalhe é decisivo: se o bootstrap for reduzido a uma chamada opaca que já faz tudo de uma vez, o seu sistema perde a capacidade de inspecionar, depurar, auditar e recuperar falhas na fronteira entre isolamento preparado e processo efetivamente iniciado. [5]

A especificação OCI também define que os namespaces Linux entram no `config.json` como uma coleção de intenções: cada entrada indica um tipo e, opcionalmente, um `path`. Se o `path` não for fornecido, o runtime deve criar um novo namespace daquele tipo; se o tipo nem aparecer, o processo deve herdar o namespace do runtime; se houver tipos duplicados, isso é erro. Essa semântica é muito superior a repassar um bitmask bruto até o topo da API. Em um sistema com DDD e boas fronteiras, o objeto de domínio precisa representar “quero um novo namespace de mount” ou “quero entrar neste namespace de rede já existente”, e somente o runtime nativo converte isso em `clone3`, `setns()` ou `unshare()`. [6][13]

2. `clone3` como superfície central do bootstrap

2.1. Definição e por que `clone3` é melhor que `clone` para este projeto

A página de manual de `clone(2)` descreve o `clone3()` como um superconjunto do `clone()` antigo, com melhorias de API relevantes para runtimes de containers: espaço para flags adicionais, separação mais limpa dos argumentos e possibilidade de declarar explicitamente o tamanho da pilha do filho. O contrato é centrado em uma estrutura `clone_args` que inclui, entre outros campos, `flags`, `pidfd`, `child_tid`, `parent_tid`, `exit_signal`, `stack`, `stack_size`, `tls`, `set_tid`, `set_tid_size` e `cgroup`. O campo `size` passado à syscall deve ser inicializado com o tamanho da estrutura conhecida pelo usuário e existe justamente para permitir extensões futuras do kernel sem quebrar a chamada. Para um projeto novo, isso é um ganho estratégico: a API nativa que você expõe ao Python pode permanecer estável enquanto o backend C++ evolui junto com o kernel. [8]

Em um runtime de containers, os campos mais importantes de `clone_args` não são todos igualmente prioritários. `flags` é o centro semântico porque carrega a intenção de isolamento, compartilhamento ou criação de recursos. `pidfd` é central para supervisão livre de race conditions por PID reciclado. `exit_signal` define como a terminação é anunciada ao pai. `cgroup` permite, em kernels modernos, nascer já no cgroup v2 de destino usando `CLONE_INTO_CGROUP`. Os campos de pilha e TLS importam mais em cenários thread-like; em bootstrap de containers, a chamada é normalmente process-like, sem `CLONE_VM`, o que permite deixar `stack` e `stack_size` em zero e evitar complexidade de alocação de pilha sob diferentes ABIs. [8]

Há ainda um ponto arquitetural sutil. Diferentemente do wrapper glibc de `clone()`, que pode receber um `fn(arg)` dependendo da arquitetura e do wrapper utilizado, `clone3()` retorna no pai e no filho como uma syscall moderna. Isso significa que o caminho do filho continuará a partir do mesmo fluxo de controle do processo chamador. Em outras palavras: se o chamador for um processo Python multithread carregando um runtime embutido, o filho herdará esse mundo e terá de atravessá-lo até chegar ao `execve()`. É justamente por isso que, no contexto deste projeto, o bootstrap deve sair do processo Python e viver em um worker nativo dedicado. [8][27][28][29]

Um benefício colateral importante de `clone3()` é a validabilidade. Como os parâmetros estão estruturados, o seu *low level design* pode ter um `CloneArgsBuilder` capaz de zero-inicializar a estrutura, preencher somente os campos semanticamente válidos, verificar combinações proibidas antes da syscall e registrar, em logs estruturados, exatamente quais bits foram enviados ao kernel. Em um sistema voltado a auditoria, isso é melhor do que ter chamadas espalhadas com inteiros mágicos e ponteiros passados em posições implícitas. [8][14][15]

2.2. Namespaces como intenção de isolamento e não como bitmask bruto

O manual `namespaces(7)` define namespace como um embrulho sobre um recurso global do sistema que faz esse recurso parecer uma instância privada para os processos membros. Essa é a base conceitual de containers em Linux. A própria OCI repete a ideia e lista, no contexto do runtime, os namespaces que um runtime deveria suportar: `pid`, `network`, `mount`, `ipc`, `uts`, `user`, `cgroup` e `time`. Para o seu projeto, isso significa que o modelo de domínio não deveria falar em “flags de clone” no nível da API pública. Ele deveria falar em “conjunto de namespaces desejados”, distinguindo criação de novos namespaces, herança do namespace do runtime e entrada em namespaces existentes via `path`. [6][7]

O namespace de `mount` isola a tabela de montagens e é indispensável assim que o sistema começa a preparar rootfs, `/proc`, `/sys`, `/dev/pts` e `/dev/shm`, que a

OCI considera parte esperada do ABI Linux no container. O detalhe perigoso aqui não é apenas criar o namespace, mas controlar a propagação de montagens. Em sistemas modernos, a propagação frequentemente está em `MS_SHARED`; por isso a documentação de `mount_namespaces(7)` sugere, para aplicações que criam um novo mount namespace com `clone()` ou `unshare()`, mudar a raiz para `MS_SLAVE|MS_REC` ou `MS_PRIVATE` a fim de impedir vazamento de eventos de mount entre namespaces. Sem essa etapa, o isolamento do container fica conceitualmente incompleto. ^{[6][11]}

O namespace de `pid` merece tratamento separado. O manual `pid_namespaces(7)` explica que o primeiro processo criado em um novo PID namespace torna-se o processo de PID 1 desse namespace e assume o papel de `init`. Se esse processo morre, o kernel mata os demais membros do namespace com `SIGKILL`, e novas criações passam a falhar. Daqui saem duas consequências de design. A primeira é que o seu bootstrap precisa ser explícito sobre quem será o PID 1 real: o próprio processo do usuário ou um `init` mínimo. A segunda é que supervisão e limpeza devem tratar a morte do PID 1 como falha estrutural do container, e não como mero término de processo. ^[10]

O namespace de `user` é o que torna `rootless` viável e o que mais exige coordenação entre pai e filho. A documentação de `user_namespaces(7)` mostra que, desde Linux 3.8, processos sem privilégios podem criar user namespaces e que, quando `CLONE_NEWUSER` é combinado com outros `CLONE_NEW*` numa mesma chamada, o user namespace é criado primeiro, dando ao filho capacidades sobre os namespaces restantes. Mas isso não elimina a necessidade de escrever os mapeamentos de IDs. O mesmo manual explica a regra de `/proc/pid/setgroups`: para muitos cenários de mapeamento não privilegiado, é preciso escrever `deny` antes de popular `gid_map`. Essa regra não é detalhe opcional; ela é parte do protocolo de bootstrap. ^{[8][9]}

O namespace de `network` isola dispositivos, pilha de rede e portas; o de `uts` permite `hostname` e domínio separados; o de `ipc` separa IPC System V e filas POSIX; o de `cgroup` virtualiza a visão da hierarquia de cgroups; o de `time` permite relógios próprios quando suportado. Do ponto de vista do desenho do sistema, o caso mais fácil é `uts`; o mais traiçoeiro, quase sempre, é a combinação de `user`, `mount` e `pid`, porque ali se concentram as decisões sobre capabilities, mapeamentos, mounts, propagação e o papel do PID 1. ^{[6][7][12]}

O `clone(2)` também impõe restrições que o seu runtime precisa validar antes de entrar no kernel. `CLONE_NEWNS` não pode ser usado com `CLONE_FS`. `CLONE_NEWPID` não pode ser combinado com `CLONE_THREAD` e, em vários casos, também conflita com `CLONE_PARENT`. `CLONE_NEWUSER` não pode ser usado com `CLONE_THREAD` nem `CLONE_PARENT`, e por razões de segurança não deve ser combinado com `CLONE_FS`. Em `clone3()`, se `CLONE_THREAD` ou `CLONE_PARENT` estiverem nas flags, um `exit_signal` não nulo gera `EINVAL`. Esses pontos não devem ficar

escondidos em documentação interna: eles precisam aparecer como regras explícitas do validador nativo e, idealmente, como mensagens de erro semânticas retornadas ao Python. [8]

2.3. `exit_signal` e o contrato de término do processo

A semântica de `exit_signal` parece simples, mas tem impacto direto sobre a supervisão. No `clone()`, o sinal de término ocupa o byte baixo de `flags`; no `clone3()`, ele ganha o campo próprio `cl_args.exit_signal`. Quando o filho termina, esse sinal pode ser enviado ao pai. Se ele for diferente de `SIGCHLD`, o pai terá de usar `__WALL` ou `__WCLONE` ao esperar pelo filho. Se for zero, o pai não recebe sinal algum na terminação. Esse desenho abre duas linhas possíveis para o seu runtime. [8]

A primeira linha, mais simples e recomendável para o MVP, é usar `exit_signal = SIGCHLD` e ainda assim requisitar `CLONE_PIDFD`. Essa combinação entrega o que há de melhor dos dois mundos: compatibilidade com ferramentas e expectativas tradicionais de espera por filho, mais uma referência estável por file descriptor para observação, `poll/epoll`, `waitid(P_PIDFD)` e sinalização posterior por `pidfd_send_signal()`. O manual de `pidfd_open(2)` é claro ao dizer que, em certas condições, a forma preferida de obter um `pidfd` de um filho é já criá-lo com `CLONE_PIDFD`, evitando janelas em que o processo já poderia ter sido reaped ou em que a política de `SIGCHLD` atrapalha. [23][24][25]

A segunda linha seria usar `exit_signal = 0` e operar em um modelo de supervisão puramente por `pidfd`. Ela é sedutora porque reduz dependência de sinais tradicionais e deixa o contrato de término mais explícito. Ainda assim, ela cobra maturidade operacional maior: toda a instrumentação, os caminhos de limpeza e a integração com ferramentas precisam aceitar que o pai não será notificado por `SIGCHLD`. Para um projeto que ainda está estabelecendo a arquitetura, a escolha de `SIGCHLD` é mais pragmática. Nada impede que um modo “`pidfd-only`” exista depois, como estratégia avançada de supervisão. [8][23][24][25]

2.4. Raw syscall wrapper: por que ele existe e onde ele deve viver

O manual `syscalls(2)` descreve a system call como a interface fundamental entre aplicação e kernel. Em geral, aplicações usam wrappers da `libc`, mas o mesmo manual ressalta que `syscall(2)` pode ser usado quando não há wrapper disponível. O manual específico de `syscall(2)` repete essa ideia e lembra que o wrapper genérico salva registradores, executa a chamada, restaura o contexto e traduz falhas para `errno`. Para um runtime de containers, esse caminho é importante não só quando faltam wrappers, mas quando se quer controlar

integralmente a ABI, a detecção de `ENOSYS`, a estratégia de fallback e o logging dos parâmetros brutos. [14][15]

Os próprios selftests do kernel para `clone3` são um excelente guia de estilo. O header de testes define um `__NR_clone3` quando necessário, constrói uma estrutura compatível e invoca `syscall(__NR_clone3, args, size)`; em seguida, usa valores deliberadamente inválidos de `exit_signal` para testar suporte e semântica. O arquivo de testes principal inicializa `exit_signal = SIGCHLD` como caso padrão e varre cenários inválidos. A lição é simples: o wrapper de `clone3` no seu C++ deve ser pequeno, determinístico e audível; a inteligência não mora dentro do wrapper, mas no `builder` e no coordenador de bootstrap que decidem como preenchê-lo. [16][17]

Esse wrapper deve viver em C++, nunca em Python. O control plane Python não deveria conhecer números de syscalls, detalhes de alinhamento de stack, bits de namespace nem políticas de retry em `EINTR`. O papel do Python é pedir “crie um container com este plano de isolamento”. O papel do C++ é decidir se isso vira `clone3` direto, `clone3 + setns`, `clone3 + fork`, ou falha por incompatibilidade do host. O Cython entra apenas como anti-corruption layer: ele converte objetos Python para DTOs nativos, chama o runtime sem GIL quando houver espera bloqueante e reconstrói exceções de domínio na volta. [21][22]

3. O que o `runc` ensina sobre a realidade do bootstrap

O melhor antídoto contra simplificações ingênuas é olhar para o `runc`. O README do `libcontainer` afirma sem rodeios que containers são gerados em um processo de duas etapas: é preciso um binário que será executado como init do container, e o projeto usa o binário atual (`/proc/self/exe`) para isso, chamando a primeira etapa de *bootstrap*. O estágio inicial é tratado via `nsenter`. Em outras palavras, o runtime de referência da OCI já organiza a criação do container como uma coreografia multiestágio, não como uma única syscall elegante. [18]

Os comentários do arquivo `nsexec.c` são ainda mais didáticos. Eles explicam que não basta “despejar as namespace flags em um clone”, porque há casos que exigem granularidade extra, como rootless containers, interações com SELinux e kernels antigos. O mesmo comentário observa que fazer `unshare(CLONE_NEWUSER)` antes de `clone()` destrói capacidades úteis do pai no namespace antigo e dificulta mapeamentos múltiplos de `uid/gid`; por isso o `runc` precisa, em muitos cenários, primeiro clonar e só depois completar a dança de namespaces e mapeamentos. A mesma passagem também lembra que para entrar corretamente em um PID namespace em certos fluxos é preciso um *double fork*, já que o efeito do PID namespace incide sobre filhos. [19]

Esses comentários se conectam diretamente ao que a documentação oficial do kernel diz. O `setns(2)` explica que, quando se entra em um PID namespace via `setns()`, o namespace do chamador não muda para o processo atual; ele muda apenas o namespace em que filhos futuros serão criados. Isso quer dizer que “criar um novo PID namespace” e “entrar em um PID namespace existente” não são o mesmo problema. O primeiro pode ser resolvido com `CLONE_NEWPID` no momento certo; o segundo exige que você tenha um processo intermediário que entra no namespace e só então crie o verdadeiro init do container. Essa distinção precisa estar gravada na sua modelagem, e não escondida em algum if obscuro do C++. [10][13][19]

O `nsexec.c` também expõe o protocolo de mapeamento do user namespace. Há uma rotina dedicada para `update_setgroups()` com o comentário explícito de que ela precisa rodar antes de tocar `gid_map`, além de rotinas para popular `uid_map` e `gid_map` com fallback para ferramentas auxiliares em caso de `EPERM`. O valor dessa referência não está em copiar o código, mas em aprender a arquitetura: mapeamentos de identidade não são uma etapa administrativa posterior, e sim parte do handshake central do bootstrap. [9][19]

Por fim, o changelog recente do `runc` mostra que a indústria de runtimes continua incorporando recursos novos do kernel, incluindo suporte ao `CLONE_INTO_CGROUP` de `clone3()` em fluxos de `exec`. Isso reforça que uma arquitetura baseada em `clone3`, `pidfd` e `cgroup v2` não é uma aposta exótica; ela segue a direção do ecossistema principal. O erro seria tentar usar essas capacidades sem reproduzir a disciplina de design que runtimes maduros precisaram construir para sobreviver a rootless, LSMs, estados intermediários e falhas parciais. [20]

4. Proposta de arquitetura para o projeto Python + Cython + C++

A arquitetura proposta parte de uma tese simples: a camada Python deve possuir o significado do sistema, e a camada C++ deve possuir o poder operacional sobre o kernel. Em termos de responsabilidade, Python guarda o contrato público, os metadados persistidos, a identificação do container, o versionamento do spec, a negociação de políticas, a autorização, a auditoria e a interpretação do ciclo de vida. O C++ guarda tudo que toca diretamente processos, namespaces, cgroups, file descriptors, sincronização entre pai e filho, signals, pidfds e preparação do ambiente de execução. O Cython, nesse arranjo, é uma barreira tipada contra vazamento de abstrações. [1][2][3][4][21][22]

A recomendação de produção é que o Cython não invoque `clone3()` a partir de um processo Python de API. A documentação oficial do Python deixa claro que chamadas equivalentes a `fork()` devem ocorrer na thread principal do

interpretador principal, e a documentação recente da linguagem passou a alertar de forma mais dura sobre a incompatibilidade estrutural entre `fork` e processos multithread. Em Python 3.14, em POSIX, o método padrão de criação de processos passou a ser `forkserver`, justamente para evitar esse tipo de problema. O paralelo conceitual é fortíssimo: também aqui o processo que chama `clone3()` deve ser um agente nativo pequeno, previsível e controlado. [27][28][29]

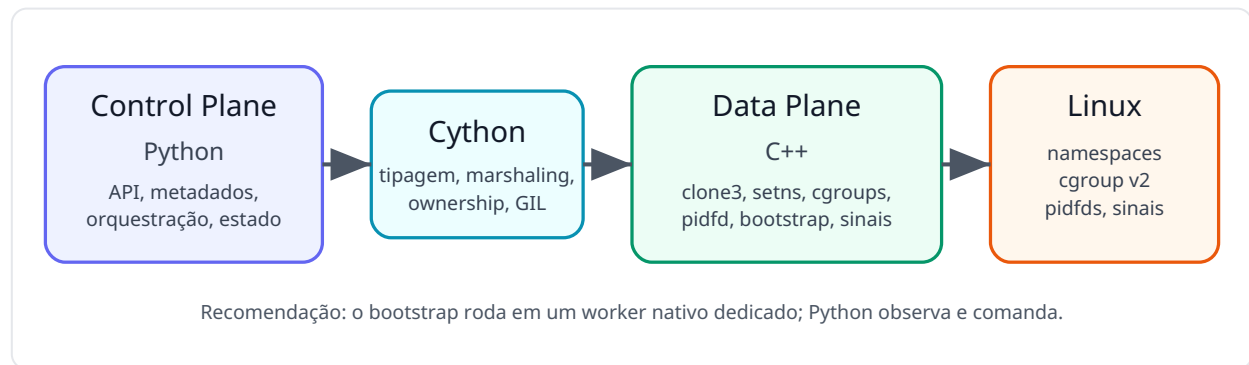


Figura 1 — Separação recomendada entre control plane, fronteira Cython, runtime nativo e kernel.

Por isso, a melhor leitura da sua exigência “Cython como cola” não é “Cython como local do bootstrap”, e sim “Cython como fronteira oficial entre Python e um runtime core nativo”. Na prática, isso pode ser implementado de duas maneiras. A maneira mais segura para produção é um worker nativo residente, single-threaded, com IPC local por UNIX domain socket, file descriptors passados por `SCM_RIGHTS` e uma biblioteca Cython que atua como cliente tipado. A maneira mais simples para laboratório é uma biblioteca C++ embutida via Cython, desde que chamada a partir de um supervisor Python dedicado e de thread única. O relatório recomenda a primeira opção como alvo principal e a segunda apenas como modo de teste. [21][22][27][28]

Em termos de componentes, o desenho fica naturalmente particionado. O componente de API em Python recebe pedidos como `CreateContainer`, `StartContainer`, `KillContainer` e `DeleteContainer`. Um componente de metadados, também em Python, persiste `spec` imutável, estado atual, capacidades negociadas, erros e timestamps. O runtime client em Cython converte o modelo Python para DTOs nativos e chama o runtime C++. O runtime C++ orquestra `clone3`, `setns()`, `unshare()`, uid/gid mappings, cgroup placement, pidfd supervision e preparação do stage de `execve()`. Por fim, um helper nativo de bootstrap — que pode ser o próprio binário reexecutado com um subcomando interno, à maneira do `runc` — executa o caminho do filho com o mínimo de dependências possível. [18][19]

Essa separação tem uma consequência importante sobre o contrato público. O Python não deveria expor diretamente “flags de clone” nem file descriptors crus.

Ele deveria expor um spec semântico, contendo algo como `namespace_intents`, `user_mappings`, `cgroup_target`, `rootfs_plan`, `process_spec` e `bootstrap_policy`. Em troca, o runtime deveria devolver um `ProcessHandle` com `pid`, `pidfd` e estados transitórios, ou um identificador opaco equivalente. Isso reduz acoplamento, permite DDD real e mantém o kernel como detalhe de implementação da camada nativa. [5][6][23][24][25]

5. System Design do subsistema de bootstrap

O primeiro ponto de system design é aceitar a separação entre `create` e `start` como uma vantagem estrutural, mesmo que a API pública externa, por conveniência, ofereça um comando único. A especificação OCI trata `create` como o momento em que o ambiente do container é materializado sem executar o programa do usuário, e `start` como o momento em que o processo final realmente começa. Reproduzir essa distinção dentro do sistema ajuda a diagnosticar falhas, a implementar hooks futuros, a manter estados coerentes e a preservar a possibilidade de recuperação após falha parcial. O design proposto, portanto, é um runtime com interface interna de duas fases: o Python pode oferecer `run`, mas por baixo dele sempre executará `create` seguido de `start`. [5]

No passo de `create`, o control plane gera um `BootstrapPlan` imutável a partir do spec do container e do conjunto de capacidades do host. Esse plano declara quais namespaces serão criados, quais serão herdados, quais serão obtidos via `setns()`, se haverá user namespace, que estratégia de PID namespace será usada, se existe cgroup v2 de destino, se `CLONE_INTO_CGROUP` é suportado, se `pidfd` será requerido e se o container terá ou não um init mínimo. O runtime nativo recebe esse plano e retorna, ao fim do `create`, um estado `created` com identidade de processo, canais de supervisão e detalhes de readiness. O `start` apenas libera a execução do processo configurado. [5][6][8][23][24][25]

O segundo ponto é a negociação de capacidades. O runtime nativo precisa possuir uma fase de `probe` na inicialização, antes mesmo de criar o primeiro container. Essa fase não é cosmética. Ela deve testar a existência de `clone3`, suporte a `CLONE_PIDFD`, suporte a `CLONE_INTO_CGROUP`, disponibilidade de user namespaces, suporte a cgroup namespace, existência e modo do cgroup v2 e regras locais de permissão. Os selftests do kernel mostram uma técnica útil: invocar `clone3` com parâmetros deliberadamente inválidos para distinguir `ENOSYS` de `EINVAL` e confirmar a presença da syscall. O resultado dessa sonda deve ser persistido pelo Python como parte do contexto do host, pois impacta a interpretação de specs e a geração de erros. [8][16][17][26]

O terceiro ponto é identidade e supervisão. Neste relatório, a recomendação é que o runtime trate o `pidfd` como identidade operacional principal do processo,

enquanto o `pid` permanece como atributo derivado e útil para interfaces humanas e compatibilidade. O motivo é robustez. O manual de `pidfd_open(2)` explica que `pidfds` são referências estáveis e observáveis por `poll/epoll`; o manual de `wait(2)` explica o suporte a `waitid(P_PIDFD)`; e `pidfd_send_signal(2)` destaca a vantagem de eliminar race conditions causadas por reciclagem de PID na sinalização de processos. Em um sistema de containers, onde o processo do container pode existir por muito tempo e o control plane pode sofrer reinícios, esse detalhe vale ouro. [23][24][25]

O quarto ponto é persistência de estado. O metastore Python deve manter, por container, o spec original, a versão normalizada do spec, o resultado do `probe` de capacidades, o estado OCI-like atual, timestamps por transição, a relação entre container ID e handles nativos, o histórico de erros e, idealmente, uma timeline de bootstrap com marcos como “clone3 emitido”, “mappings solicitados”, “mappings aplicados”, “namespace ready”, “process started”. Isso viabiliza observabilidade, auditoria, correlação de falhas e reconstituição de contexto após crash do control plane. [5][18][19]

O quinto ponto é modelar explicitamente a diferença entre visibilidade e pertencimento em cgroups. O cgroup namespace virtualiza a visão do caminho de cgroup, mas não altera por si só a associação do processo aos control groups. Já o `CLONE_INTO_CGROUP` nasce justamente para colocar o filho diretamente no cgroup v2 de destino e evitar jitter de accounting e o custo de mover o processo depois da criação. O design, portanto, precisa distinguir dois conceitos no plano: `cgroup_visibility` e `cgroup_placement`. Misturar ambos em um único campo é receita para bugs difíceis de interpretar. [8][12][26]

6. Low Level Design do bootstrap

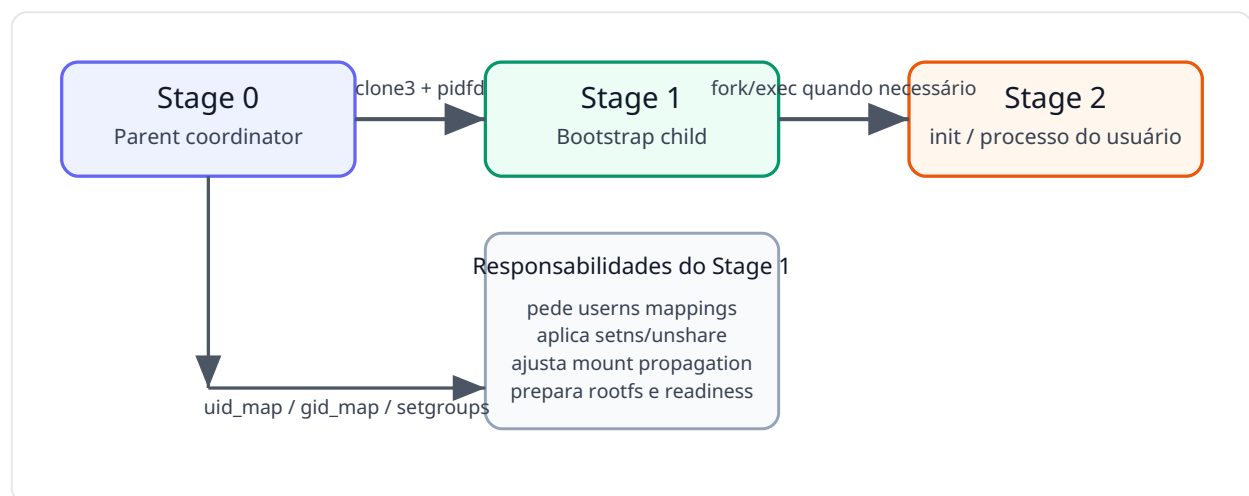


Figura 2 — Fluxo recomendado do bootstrap em estágios com handshake de user namespace.

O low level design recomendado começa com um conjunto de tipos nativos coesos. Um `KernelFeatureProbe` representa as capacidades do host e sabe testá-las de forma determinística. Um `NamespaceIntent` representa um tipo de namespace e o modo desejado: *new*, *inherit* ou *join(path)*. Um `UserMappingPlan` representa a política de `uid_map`, `gid_map` e `setgroups`. Um `CgroupPlacement` representa a intenção de nascer em um cgroup específico ou mover-se depois. Um `ExitPolicy` encapsula o valor de `exit_signal` e o modo de supervisão. Por cima deles, um `BootstrapPlan` agrega a estratégia completa para o container. Essa modelagem é superior a passar um grande struct plano com dezenas de campos opcionais sem semântica. [6][8][9][12]

O wrapper bruto de `clone3` deve ser minúsculo. A função idealmente recebe um objeto já validado, zera uma `clone_args`, preenche `flags`, `pidfd`, `exit_signal`, `cgroup` e, se necessário, campos de stack, e então chama `syscall(SYS_clone3, &args, sizeof(args))`. Todo o resto fica fora dali: decisão de flags, montagem do cgroup fd, preparação de socketpairs, abertura de namespace fds, logging e rastreamento de falhas. O wrapper deve sempre registrar o erro nativo em `errno`, convertendo-o em uma exceção rica do domínio nativo, por exemplo `UnsupportedKernelFeature`, `InvalidNamespaceCombination`, `UserMappingFailure` ou `CgroupPlacementFailure`. [8][14][15][16][17]

O coordenador do bootstrap, chamado aqui de `Stage0Parent`, é quem prepara os canais de sincronização. A referência do `runc` mostra o uso de `socketpair()` para sincronizar pai, filho e às vezes neto; a ideia fundamental é que o filho não prossiga para etapas irreversíveis enquanto o pai não confirmar que mapeamentos ou recursos externos foram aplicados. Em um design moderno, um par de sockets UNIX full-duplex é suficiente para transportar mensagens de estado, `errno` serializado, logs de bootstrap e, quando necessário, file descriptors por `SCM_RIGHTS`. O pai chama `clone3` com `CLONE_PIDFD`, retém o `pidfd` do filho, e entra em uma pequena máquina de estados. [19][23][24]

O `Stage1BootstrapChild` nasce em um contexto o mais estreito possível. Ele não deve chamar de volta o interpretador Python. Seu trabalho é executar ações de isolamento e então tornar-se o processo final ou preparar o `Stage2`. Se houver user namespace com mapeamento explícito, o filho emite um evento `REQUEST_USERMAP`, fica bloqueado no socket e aguarda o pai escrever `/proc/<pid>/setgroups`, `uid_map` e `gid_map`. Só depois disso continua. Se houver mount namespace novo, deve aplicar a política de propagação logo no início, idealmente `MS_SLAVE|MS_REC` ou `MS_PRIVATE`, e em seguida materializar os mounts esperados pelo ABI do container, conforme o spec OCI. [6][9][11][19]

A estratégia de PID namespace depende do caso de uso. Quando a intenção é criar um novo PID namespace a partir do zero, usar `CLONE_NEWPID` diretamente no `clone3` do filho pode ser suficiente, e o filho já nasce como PID 1 no novo namespace. Quando a intenção é entrar em um PID namespace existente via

`setns()`, isso não basta, porque o `setns()` só afeta os filhos futuros. Nesse cenário, o `Stage1BootstrapChild` precisa entrar no namespace e então criar o verdadeiro `Stage2`, que será o init do container. O low level design deve tratar os dois fluxos como estratégias distintas, e não como branches acidentais do mesmo método. [8][10][13][19]

O `Stage2` representa o processo que sobreviverá à fase de bootstrap. Dependendo da política do sistema, ele pode ser um init mínimo escrito em C++ que reencaminha sinais, reape zombies e então executa o processo do usuário, ou pode ser o próprio processo do usuário, quando se decidir que o `entrypoint` será PID 1. O relatório recomenda oferecer as duas opções no modelo, mas implementar primeiro o modo “processo do usuário como PID 1”, complementado por um init mínimo opcional para workloads que precisem de reaping robusto. Isso se alinha ao fato de que o primeiro processo do PID namespace assume responsabilidades de init e à necessidade de não mascarar essas responsabilidades do usuário do runtime. [10]

A supervisão deve ser `pidfd-first`. Depois da criação, o pai observa o `pidfd` via `epoll` ou `waitid(P_PIDFD)`. Para término normal, ele captura código de saída e atualiza o estado para `stopped`. Para interrupção, usa `pidfd_send_signal()` preferencialmente a `kill(pid)`. Se o runtime precisar reentrar em namespaces de outro processo para operações futuras, a API `setns()` aceita, desde Linux 5.8, um `pidfd` como origem para entrar atômica e seletivamente em vários namespaces do processo-alvo. Isso abre espaço para `attach` e `exec` mais seguros no futuro. [13][23][24][25]

Um ponto muitas vezes negligenciado é `cgroup v2`. O low level design deve preferir abrir o diretório do `cgroup` alvo com `O_PATH` ou `O_RDONLY`, preencher `cl_args.cgroup` e ativar `CLONE_INTO_CGROUP` quando houver suporte. Isso permite que o filho nasça já no `cgroup` correto, evitando jitter de accounting e o custo de mover um processo após a criação. Ainda assim, o runtime precisa saber degradar quando o recurso não estiver disponível, quando o host não usar `cgroup v2` ou quando o `cgroup` estiver em estado inválido de domínio/threaded topology. Nesses casos, a política deve ser explícita: falhar cedo ou cair para “spawn and move” dependendo do perfil de consistência desejado. [8][20][26]

Na fronteira Cython, a regra é simples. Declarações de C++ devem morar em arquivos `.pxd` com `cdef extern from` e `cppclass`; o módulo `.pyx` faz o marshaling dos objetos Python, invoca o C++ e reconstrói respostas Python. Construtores e chamadas que possam lançar exceção C++ devem ser declarados com `except +`, para que o erro atravesse a fronteira como exceção Python apropriada. Chamadas bloqueantes ou loops de espera devem rodar em `with nogil`, porque Cython explicita que código `nogil` não pode tocar objetos Python e é esse o comportamento desejado aqui. O GIL só volta no instante de converter o resultado ou o erro para o mundo Python. [21][22]

6.1. Sketch de implementação para orientar a construção

Os trechos abaixo não pretendem ser uma implementação pronta; servem como âncoras de design. O objetivo é mostrar o formato das fronteiras e o tipo de responsabilidade que pertence a cada camada. O C++ fica com o wrapper e o protocolo do bootstrap. O Cython traduz tipos e libera o GIL durante chamadas bloqueantes. O Python persiste estado e orquestra o lifecycle. [8][14][21][22][27][28]

Exemplo conceitual de wrapper e plano nativo em C++

```
// bootstrap_clone3.hpp
struct NamespaceIntent {
    enum class Type { User, Mount, Pid, Net, Uts, Ipc, Cgroup, Time };
    enum class Mode { New, Inherit, JoinPath };
    Type type;
    Mode mode;
    std::string path;    // válido apenas em JoinPath
};

struct BootstrapPlan {
    std::vector<NamespaceIntent> namespaces;
    bool request_pidfd = true;
    int exit_signal = SIGCHLD;
    int target_cgroup_fd = -1;
    bool rootless = false;
    bool use_minimal_init = false;
};

class Clone3Invoker {
public:
    pid_t Spawn(const BootstrapPlan& plan, int* pidfd_out) {
        struct clone_args args {};
        args.flags = BuildFlags(plan);
        args.exit_signal = static_cast<uint64_t>(plan.exit_signal);

        if (plan.request_pidfd) {
            args.flags |= CLONE_PIDFD;
            args.pidfd = reinterpret_cast<uint64_t>(pidfd_out);
        }
        if (plan.target_cgroup_fd >= 0) {
            args.flags |= CLONE_INTO_CGROUP;
            args.cgroup = static_cast<uint64_t>(plan.target_cgroup_fd);
        }

        long rc = syscall(SYS_clone3, &args, sizeof(args));
        if (rc == -1) throw SysError("clone3");
        return static_cast<pid_t>(rc);
    }
};
```

No trecho C++, o detalhe mais importante é o que não aparece: nenhuma lógica de domínio do control plane e nenhum contato com objetos Python. O wrapper

recebe um plano já validado, constrói `clone_args` e chama a `syscall`. O cálculo de flags fica encapsulado em `BuildFlags`, que é precisamente o lugar em que a implementação deve rejeitar combinações proibidas, interpretar `rootless/rootful` e decidir se o fluxo será direto ou multiestágio. [8][14][15]

Exemplo conceitual da fronteira Cython

```
# runtime.pxd
cdef extern from "bootstrap_api.hpp" namespace "dkrt":
    cdef cppclass ProcessHandle:
        int pid
        int pidfd
    cdef cppclass BootstrapSpec:
        BootstrapSpec() except +
        ProcessHandle create_container(BootstrapSpec&) except +

# runtime.pyx
cdef BootstrapSpec marshal_spec(object py_spec) except *:
    cdef BootstrapSpec spec = BootstrapSpec()
    # preencher campos nativos a partir do spec Python
    return spec

def create_container_py(py_spec):
    cdef BootstrapSpec spec = marshal_spec(py_spec)
    cdef ProcessHandle handle
    with nogil:
        handle = create_container(spec)
    return {"pid": handle.pid, "pidfd": handle.pidfd}
```

Na fronteira Cython, a documentação oficial recomenda declarar classes C++ com `cppclass` e usar `except +` para traduzir exceções nativas em exceções Python. A região `with nogil` é o lugar natural para chamadas que podem bloquear em sincronização, espera por readiness ou IPC com o worker nativo. A volta ao Python só deve ocorrer depois que o resultado estiver materializado em tipos triviais ou que uma exceção de domínio já exista. [21][22]

Exemplo conceitual do comando de aplicação em Python

```
class CreateContainerHandler:
    def __init__(self, repo, runtime):
        self.repo = repo
        self.runtime = runtime

    def handle(self, command):
        container = self.repo.allocate(command.container_id, command.spec)
        self.repo.mark_creating(container.id)
        try:
            result = self.runtime.create_container(container.spec)
            self.repo.mark_created(
                container.id,
                pid=result["pid"],
                pidfd=result["pidfd"],
            )
            return container.id
        except Exception as exc:
            self.repo.mark_failed(container.id, reason=str(exc))
            raise
```

Já no Python, o comando é pura orquestração e persistência. Essa separação é importante para o DDD e para a confiabilidade operacional: o handler não conhece syscalls, mas conhece estados de negócio do runtime, decide persistir `creating`, marca `created` ou `failed` e preserva a auditabilidade do processo. O mesmo padrão se aplica a `start`, `kill` e `delete`. [5][21][22]

7. Domain-Driven Design aplicado ao bootstrap

O risco mais comum ao introduzir DDD em infraestrutura de containers é modelar diretamente a API do kernel como “domínio”. Isso é um erro. O domínio do seu sistema não é `clone3`; o domínio é “materializar e supervisionar um sandbox de execução conforme uma intenção declarativa de isolamento”. `clone3`, `setns()`, `pidfd` e `cgroups` são mecanismos do subdomínio técnico de runtime. O DDD entra justamente para impedir que esses mecanismos vazem para o modelo conceitual da aplicação. [5][6][8]

O bounded context de Control Plane, implementado majoritariamente em Python, deve conter entidades como `Container`, `ContainerSpec`, `LifecycleState` e `HostCapabilityProfile`. O bounded context de Runtime Bootstrap, implementado em C++, deve conter entidades ou agregados como `BootstrapSession`, `ProcessHandle` e `SandboxInstance`. Entre os dois existe uma anti-corruption layer implementada pelo Cython. Essa camada converte o vocabulário do domínio em DTOs nativos e traduz exceções, evitando que o control plane passe a depender da ABI do kernel ou que o runtime passe a carregar conceitos de API externa e persistência. [21][22]

O aggregate root mais importante do lado Python continua sendo o `Container`, porque é ele que agrega identidade, spec, estado e histórico. Mas do lado nativo é útil introduzir um aggregate especializado chamado `BootstrapSession`. Ele encapsula a execução única do protocolo de bootstrap, incluindo `BootstrapPlan`, canais de sync, estado interno de cada stage, pidfd, erros nativos e recursos temporários. Isso separa claramente o “container como entidade de negócio do runtime” do “bootstrap como transação técnica de criação”. Sem essa distinção, erros de preparação de ambiente acabam contaminando o lifecycle inteiro do objeto container e gerando modelos confusos. [5][18][19]

Os value objects centrais deste subdomínio são bem definidos. `NamespaceIntent` é um value object, pois sua identidade é dada pela combinação de tipo e modo de aplicação. `UserMappingPlan` também, porque representa uma regra imutável de tradução de IDs. `CgroupPlacement`, `ExitPolicy`, `KernelFeatureSet` e `RootfsPlan` seguem a mesma lógica. Todos eles devem ser imutáveis depois de construídos, o que torna o bootstrap reexecutável em testes, serializável para auditoria e comparável entre hosts. [6][8][9][26]

As operações de orquestração que não pertencem a uma única entidade devem virar domain services. `BootstrapPlanner` recebe `ContainerSpec` e `HostCapabilityProfile` e produz um `BootstrapPlan`. `NamespaceCompatibilityPolicy` valida combinações proibidas de namespaces e flags. `ProcessSupervisionService` traduz eventos de pidfd em mudanças de estado do container. `UserMappingService` coordena escrita de `setgroups`, `uid_map` e `gid_map` no momento certo. Esses serviços podem ser implementados em C++ ou Python conforme a natureza da regra, mas sua assinatura deve permanecer semântica, não orientada a syscalls. [8][9][10][23][24][25]

Os eventos de domínio também ajudam muito. Eventos como `ContainerBootstrapRequested`, `BootstrapChildSpawned`, `UserMappingsApplied`, `ContainerEnvironmentCreated`, `ContainerStarted`, `ContainerExited` e `BootstrapFailed` tornam o fluxo legível e facilitam observabilidade. Eles não são meros logs. Eles são parte da gramática da aplicação. Em especial, ao registrar `BootstrapFailed` com causa estruturada, o control plane consegue diferenciar um erro de spec inválido, um erro de permissão do host, uma incompatibilidade de kernel e uma falha do próprio processo do usuário. [5][19]

8. Padrões de Design GoF adequados a esta feature

O primeiro padrão que se destaca é o *Facade*. O Cython deve expor para o Python uma fachada pequena e estável, algo como `RuntimeClient.create(spec)`, `start(id)`, `kill(id, signal)` e `delete(id)`. O objetivo não é simplificar só por estética, e sim impedir que a aplicação em

Python veja dezenas de conceitos nativos de baixo nível. Essa fachada define a superfície oficial entre os mundos e evita acoplamento indevido à implementação C++. [21][22]

O segundo é o *Builder*. Tanto `BootstrapPlanBuilder` quanto `CloneArgsBuilder` são casos claros. O plano de bootstrap nasce de várias decisões derivadas: namespaces novos ou herdados, estratégia de PID, política de cgroup, `pidfd`, `exit_signal`, `rootless` ou `rootful`. Já a estrutura `clone_args` precisa ser construída de modo seguro, zero-inicializado e validado. Um builder é preferível a construtores gigantes ou mutação ad hoc porque a ordem das decisões importa e o objeto final só deveria existir quando estiver semanticamente completo. [8]

O terceiro é o *Strategy*. Várias partes do bootstrap são, na prática, famílias de algoritmos substituíveis. A aplicação de namespaces pode seguir uma `CreateNamespaceStrategy` ou uma `JoinNamespaceStrategy`. A supervisão pode usar uma `PidfdSupervisionStrategy` como padrão e um fallback restrito quando `pidfds` não estiverem disponíveis. A colocação em cgroup pode seguir uma `CloneIntoCgroupStrategy` ou uma `SpawnThenMoveStrategy`. A escolha de strategy com base em `KernelFeatureSet` torna o sistema mais claro que espalhar condicionais por toda a base nativa. [8][23][24][25][26]

O quarto é o *Abstract Factory*. `Rootful`, `rootless`, `join` de namespaces existentes e criação de namespaces novos não são variações cosméticas; são famílias inteiras de objetos e passos. Uma fábrica abstrata de pipelines de bootstrap pode devolver um conjunto coeso de componentes compatíveis entre si, por exemplo um pipeline `rootless` com `UserMappingService` habilitado e um pipeline `rootful` simplificado. Isso reduz acoplamento entre escolhas de ambiente e construção concreta dos objetos. [9][18][19]

O quinto é o *State*. O ciclo `creating` → `created` → `running` → `stopped` da OCI não é só um enum; ele altera quais operações são válidas e quais devem falhar. Um objeto de estado ou, no mínimo, uma máquina de estados explícita impede que chamadas inválidas como `start` em container não criado ou `delete` em container em execução virem “ifs” casuais espalhados. Aqui, o padrão tem aderência direta à semântica da especificação OCI. [5]

O sexto é o *Command*. Operações como criar, iniciar, matar, executar um processo extra ou deletar podem ser tratadas como comandos explícitos, serializáveis e auditáveis. Isso combina bem com o control plane Python, que pode persistir a intenção do comando antes de chamá-lo e registrar o resultado depois. Em sistemas distribuídos ou assíncronos, esse padrão ainda facilita reexecução idempotente e compensações. [1][5]

O sétimo é o *Template Method*. A sequência geral do bootstrap — validar plano, preparar recursos, clonar, sincronizar mapeamentos, aplicar namespaces, preparar mounts, materializar cgroup, tornar processo pronto, executar

`execve()` — tende a permanecer fixa, mas alguns passos variam conforme `rootless`, `join` de PID namespace ou presença de `cgroup target`. Um `template method` na camada nativa ajuda a preservar a ordem do protocolo sem impedir especializações controladas. [18][19]

O oitavo é o *Observer*. A combinação de eventos de domínio, logs estruturados e supervisão por `pidfd` pede um mecanismo de assinatura para mudança de estado. `Control plane`, `telemetria`, `auditoria` e eventualmente um CLI de debugging podem observar a mesma sequência de eventos. Isso também se encaixa bem em um modelo onde o runtime worker publica eventos internos e o Python os traduz para persistência e API. [23][24][25]

9. Requirements funcionais e não funcionais

9.1. Requirements funcionais

RF-1 — Criação determinística e validada do processo filho

A implementação deve aceitar um spec declarativo de bootstrap e transformá-lo em uma criação determinística do filho via `clone3()`, validando previamente combinações ilegais de flags, suporte do kernel e coerência entre namespaces, `cgroup target` e política de sinalização. Falhas de validação não devem chegar ao kernel quando puderem ser detectadas antes. [8][14][15]

RF-2 — Suporte semântico a criação, herança e entrada em namespaces

O runtime deve suportar, ao menos para os namespaces OCI principais, três modos: criar um namespace novo, herdar o namespace do runtime ou entrar em um namespace existente indicado por path absoluto. O contracto precisa diferenciar claramente esses modos e retornar erro para tipos duplicados ou incompatíveis, em aderência ao que a OCI define. [6][13]

RF-3 — Fluxo rootless/rootful com user namespace e mappings

A implementação deve suportar tanto execução privilegiada quanto `rootless`, incluindo criação de user namespace, sincronização entre pai e filho para escrita de `setgroups`, `uid_map` e `gid_map`, e mensagens de erro que indiquem com precisão se a falha ocorreu por política do host, ausência de suporte do kernel ou erro de configuração do spec. [8][9][19]

RF-4 — Supervisão robusta por `pidfd`

O resultado de um bootstrap bem-sucedido deve incluir um identificador estável para supervisão do processo, preferencialmente um `pidfd`, além do PID visível externamente. O runtime deve conseguir observar saída, enviar sinais e detectar morte do processo sem depender exclusivamente de `kill(pid)` e sem sofrer com reciclagem de PID. [23][24][25]

RF-5 — Lifecycle compatível com semântica OCI-like

O runtime deve distinguir, internamente, os estados equivalentes a `creating`, `created`, `running` e `stopped`, além de oferecer operações de `create`, `start`, `kill` e `delete` coerentes com esses estados. Ainda que a API externa consolide `create+start` em um único comando, a implementação interna deve preservar a separação. [5]

RF-6 — Colocação em cgroup v2 e isolamento de visibilidade

A implementação deve conseguir, quando suportado, criar o processo já no cgroup v2 de destino usando `CLONE_INTO_CGROUP` e também, independentemente disso, configurar a visibilidade via cgroup namespace quando tal isolamento fizer parte do spec. O sistema deve distinguir as duas dimensões para evitar inconsistência semântica. [8][12][20][26]

RF-7 — Limpeza e compensação em falhas parciais

Se o filho morrer antes do `execve()`, se os mappings falharem, se o cgroup target for inválido ou se um namespace não puder ser aplicado, o runtime deve desfazer recursos temporários, fechar file descriptors, marcar o bootstrap como falho e deixar o metastore em estado coerente, preferencialmente como se a operação nunca tivesse sido concluída. [5][19]

RF-8 — Observabilidade de bootstrap

Cada fase do bootstrap deve produzir eventos e métricas suficientemente detalhados para reconstruir o caminho da criação. Isso inclui, no mínimo, timestamps por estágio, erros estruturados com `errno` original, processo pai/filho envolvido e decisão de estratégia utilizada. Em um projeto educacional ou de pesquisa, essa observabilidade também serve como mecanismo de aprendizagem e validação. [18][19][23][24]

9.2. Requirements não funcionais

RNF-1 — Segurança por menor privilégio

O runtime deve minimizar o escopo de privilégios, manter a maior parte da lógica privilegiada fora do control plane e reduzir o caminho do filho a operações estritamente necessárias até o `execve()`. Sempre que possível, o processo Python não deve carregar privilégios ou executar syscalls de isolamento diretamente. [9][27][28][29]

RNF-2 — Compatibilidade explícita com versões e capacidades do kernel

O sistema não pode assumir suporte universal a `clone3`, user namespaces, cgroup namespaces ou `CLONE_INTO_CGROUP`. Ele deve detectar, registrar e expor esse perfil de capacidades, para que o comportamento seja previsível.

Compatibilidade implícita ou tentativa cega de uso de features novas é inaceitável em runtime de containers. [8][16][17][26]

RNF-3 — Desempenho e baixo overhead no caminho quente

O caminho de bootstrap deve minimizar cópias, alocações e mudanças de contexto. O uso de `CLONE_INTO_CGROUP` quando possível, `pidfds` em vez de polling por PID e uma fronteira Cython enxuta ajudam a reduzir overhead. Ainda assim, clareza e segurança devem ter precedência sobre micro-otimizações prematuras. [8][20][23][24][26]

RNF-4 — Robustez sob falhas e reinícios

O control plane deve conseguir reiniciar e reconstruir o estado dos containers a partir de metadados persistidos e handles observáveis. O runtime nativo deve tolerar morte precoce do filho, interrupções por sinal e falhas de sincronização sem deixar lixo operacional irreparável. [5][23][24][25]

RNF-5 — Testabilidade e repetibilidade

O desenho deve permitir testes unitários de validação de plano, testes de integração com kernel real e testes de falha por injeção. Para isso, o planner, o builder de `clone_args` e o state machine do bootstrap precisam ser componentes isoláveis, não uma massa única de código procedural. [8][16][17][19]

RNF-6 — Observabilidade operacional

Logs estruturados, métricas de tempo por etapa, eventos de lifecycle e correlação por container ID devem ser requisitos básicos, não acessórios. Sem isso, qualquer tentativa de depurar rootless, PID 1, cgroups e namespaces simultaneamente vira arqueologia de processos. [18][19][23][24]

RNF-7 — Estabilidade de API entre Python, Cython e C++

A fronteira entre Python e C++ deve evoluir por versionamento explícito de DTOs e contratos, evitando exposição direta de structs de kernel ou dependência de layout ABI instável no lado Python. O uso de classes C++ declaradas em `.pxd` e conversões controladas em `.pyx` ajuda a preservar esse requisito. [21][22]

RNF-8 — Portabilidade razoável entre arquiteturas Linux

O runtime deve ser escrito assumindo Linux, mas não apenas um kernel e uma arquitetura específica. Questões como alinhamento de stack, disponibilidade de syscalls e comportamento de wrappers devem ser tratadas com probes e camadas finas de adaptação, para permitir x86_64 e arm64 sem divergência acidental de semântica. [8][14][15]

10. Plano incremental de implementação

A implementação deveria começar por uma fase zero de infraestrutura do runtime, antes de qualquer container “real”. Nela, o objetivo é estabelecer o worker nativo, o contrato Cython, o `KernelFeatureProbe`, o tipo `BootstrapPlan`, a taxonomia de erros e o mecanismo de logs estruturados. Sem essa base, toda funcionalidade futura ficará acoplada à primeira forma improvisada de chamar `clone3()`. [16][17][21][22][27][28]

Na fase um, o sistema deve implementar um fluxo rootful mínimo com `clone3()`, `CLONE_PIDFD`, `exit_signal = SIGCHLD` e namespaces menos controversos, como `uts`, `ipc` e `mount`. O resultado esperado é um container cujo ambiente é criado em estado `created`, com `pidfd` funcional, logs por fase e operação de `start` separada. Essa fase ainda não precisa resolver rootless nem join de namespaces existentes. [5][6][8][23][24][25]

Na fase dois, entra o user namespace. É aqui que o bootstrap deixa de ser apenas spawn e passa a ser realmente um protocolo. Devem ser implementados os estados de sincronização para escrita de `setgroups`, `uid_map` e `gid_map`, os caminhos de fallback ou erro para cenários de permissão insuficiente e a diferença explícita entre rootful e rootless. Essa fase deve vir acompanhada de testes de falha que matem o filho antes da conclusão do handshake e validem a limpeza do pai. [9][19]

Na fase três, o sistema deve tratar profundamente PID namespace, inclusive a decisão entre criar um namespace novo e entrar em um namespace existente. A semântica de “processo atual versus filhos futuros” precisa virar código de produção. Aqui também vale introduzir, se desejado, um init mínimo opcional para processos que queiram delegar reaping e propagação de sinais. [10][13][19]

Na fase quatro, entra a integração mais rica com cgroups e, se a meta for um sistema mais próximo de OCI, a materialização de rootfs, mounts padrão e demais aspectos do ABI Linux exigido pelo container. O uso de `CLONE_INTO_CGROUP` deve ser habilitado quando suportado, mas com feature flag clara e fallback observável. Nesta fase também faz sentido começar a desenhar o contrato de `exec` em containers já existentes, que pode se beneficiar de `pidfds` e `setns()` atômico em kernels mais novos. [6][8][12][13][20][26]

Na fase cinco, o foco passa a ser endurecimento. Entram testes de compatibilidade em múltiplos kernels, estresse de criação paralela, falhas de sincronização, auditoria de logs, verificação de vazamento de file descriptors, simulação de crash do control plane e validação da reconstrução de estado. O objetivo dessa fase não é adicionar features, e sim transformar um protótipo funcional em um subsistema confiável. [5][18][19][23][24][25]

11. Estratégia de testes e análise de falhas

Um runtime desse tipo precisa ser testado em três níveis. O primeiro é o nível unitário, onde o foco está em geração de plano, validação de combinações ilegais, serialização de DTOs, tradução de exceções e máquina de estados. Tudo que puder ser testado sem chamar o kernel deve ser isolado e coberto aqui. O segundo é o nível de integração, com kernel real, namespaces reais, cgroups reais e user mappings reais. O terceiro é o nível de comportamento do sistema, validando transições de estado, eventos persistidos e capacidade do control plane de sobreviver a reinícios. [5][8][9][19]

Entre os casos de falha obrigatórios, destacam-se os seguintes. Um plano que pede combinações proibidas de flags deve falhar antes da syscall. Um host sem `clone3` deve produzir erro claro de incompatibilidade ou acionar fallback explícito, nunca comportamento indefinido. Um filho que morre entre o `clone3` e a escrita de `uid_map/gid_map` deve ser detectado e limpo. Um cgroup v2 em estado inválido para `CLONE_INTO_CGROUP` deve produzir erro inteligível. Um PID 1 que sai cedo precisa ser interpretado como colapso do namespace, e não como simples término benigno. [8][10][19][20][26]

Os testes de observabilidade também são essenciais. Cada caso de teste de integração deveria verificar não só o comportamento funcional, mas a emissão de eventos e métricas esperadas. Em um sistema como este, um teste que “passa” funcionalmente, mas não informa em que etapa o filho morreu, é um teste incompleto. A própria timeline de bootstrap deve ser encarada como parte do contrato do sistema. [18][19][23][24]

Por fim, vale destacar que o conjunto de testes precisa refletir o suporte real do host. A suíte deve ser capability-aware. Testes que dependem de user namespaces, cgroup namespaces ou `CLONE_INTO_CGROUP` devem pular com justificativa quando a feature não existir, mas registrar claramente a matriz de suporte do ambiente. Isso evita falsos negativos e transforma a suíte em ferramenta de diagnóstico de portabilidade. [8][16][17][26]

12. Conclusão

Bootstrap com `clone3` não é um detalhe de implementação; é o núcleo mecânico de um runtime de containers. A combinação de `clone3`, namespaces, `exit_signal`, `pidfds` e cgroup v2 oferece uma base tecnicamente superior para um sistema novo em Linux, mas só produz um resultado robusto quando é cercada por um desenho arquitetural disciplinado. O Docker original, o ecossistema Moby, o `containerd`, o `runc` e a OCI apontam todos na mesma direção: separar control plane de runtime, tratar lifecycle como contrato formal e aceitar que o bootstrap é multiestágio. [1][2][3][4][5][18][19][20]

Para o seu projeto específico, a decisão mais importante é proteger o Python do que o kernel exige de mais sensível. Python deve modelar intenção e estado; C++ deve operar o isolamento; Cython deve ser a ponte controlada entre ambos. Dentro desse arranjo, o caminho recomendado é um worker nativo dedicado, `pidfd` como identidade operacional, `exit_signal = SIGCHLD` no MVP, duas fases internas de lifecycle e um modelo de domínio em que namespaces são intenções semânticas, não inteiros mágicos. [8][21][22][23][24][25][27][28][29]

Se esse desenho for seguido, a feature “Bootstrap com `clone3`” deixa de ser apenas a primeira syscall do runtime e passa a ser a fundação sobre a qual o resto do sistema poderá crescer: rootfs, networking, exec adicional, seccomp, capabilities, snapshotters, observabilidade e até compatibilidade mais ampla com OCI. Se ele for ignorado e reduzido a um wrapper ad hoc, o projeto ainda pode funcionar em demos simples, mas pagará a conta em comportamento indefinido, debug opaco e acoplamento arquitetural prematuro. [5][6][8][18][19]

Referências

1. [1] *Docker Docs — Docker Engine*. Disponível em: <https://docs.docker.com/engine/>. Acesso em 12 mar 2026.
2. [2] *Docker Docs — Alternative container runtimes*. Disponível em: <https://docs.docker.com/engine/daemon/alternative-runtimes/>. Acesso em 12 mar 2026.
3. [3] *containerd — README*. Disponível em: <https://github.com/containerd/containerd>. Acesso em 12 mar 2026.
4. [4] *Moby Project — README*. Disponível em: <https://github.com/moby/moby>. Acesso em 12 mar 2026.
5. [5] *Open Container Initiative — Runtime and Lifecycle (runtime.md)*. Disponível em: <https://github.com/opencontainers/runtime-spec/blob/main/runtime.md>. Acesso em 12 mar 2026.
6. [6] *Open Container Initiative — Linux Configuration (config-linux.md)*. Disponível em: <https://github.com/opencontainers/runtime-spec/blob/main/config-linux.md>. Acesso em 12 mar 2026.
7. [7] *Linux man-pages — namespaces(7)*. Disponível em: <https://man7.org/linux/man-pages/man7/namespaces.7.html>. Acesso em 12 mar 2026.
8. [8] *Linux man-pages — clone(2)*. Disponível em: <https://man7.org/linux/man-pages/man2/clone.2.html>. Acesso em 12 mar 2026.
9. [9] *Linux man-pages — user_namespaces(7)*. Disponível em: https://man7.org/linux/man-pages/man7/user_namespaces.7.html. Acesso em 12 mar 2026.
10. [10] *Linux man-pages — pid_namespaces(7)*. Disponível em: https://man7.org/linux/man-pages/man7/pid_namespaces.7.html. Acesso em 12 mar 2026.
11. [11] *Linux man-pages — mount_namespaces(7)*. Disponível em: https://man7.org/linux/man-pages/man7/mount_namespaces.7.html. Acesso em 12 mar 2026.

12. [12] *Linux man-pages — cgroup_namespaces(7)*. Disponível em: https://man7.org/linux/man-pages/man7/cgroup_namespaces.7.html. Acesso em 12 mar 2026.
13. [13] *Linux man-pages — setns(2)*. Disponível em: <https://man7.org/linux/man-pages/man2/setns.2.html>. Acesso em 12 mar 2026.
14. [14] *Linux man-pages — syscall(2)*. Disponível em: <https://man7.org/linux/man-pages/man2/syscall.2.html>. Acesso em 12 mar 2026.
15. [15] *Linux man-pages — syscalls(2)*. Disponível em: <https://man7.org/linux/man-pages/man2/syscalls.2.html>. Acesso em 12 mar 2026.
16. [16] *Linux kernel selftests — clone3_selftests.h*. Disponível em: https://raw.githubusercontent.com/torvalds/linux/master/tools/testing/selftests/clone3/clone3_selftests.h. Acesso em 12 mar 2026.
17. [17] *Linux kernel selftests — clone3.c*. Disponível em: <https://raw.githubusercontent.com/torvalds/linux/master/tools/testing/selftests/clone3/clone3.c>. Acesso em 12 mar 2026.
18. [18] *runc — libcontainer README*. Disponível em: <https://github.com/opencontainers/runc/blob/main/libcontainer/README.md>. Acesso em 12 mar 2026.
19. [19] *runc — libcontainer/nsenter/nsexec.c*. Disponível em: <https://github.com/opencontainers/runc/blob/main/libcontainer/nsenter/nsexec.c>. Acesso em 12 mar 2026.
20. [20] *runc — CHANGELOG*. Disponível em: <https://github.com/opencontainers/runc/blob/main/CHANGELOG.md>. Acesso em 12 mar 2026.
21. [21] *Cython Documentation — Using C++ in Cython*. Disponível em: https://cython.readthedocs.io/en/latest/src/userguide/wrapping_CPlusPlus.html. Acesso em 12 mar 2026.
22. [22] *Cython Documentation — Interfacing with External C Code*. Disponível em: https://cython.readthedocs.io/en/latest/src/userguide/external_C_code.html. Acesso em 12 mar 2026.
23. [23] *Linux man-pages — pidfd_open(2)*. Disponível em: https://man7.org/linux/man-pages/man2/pidfd_open.2.html. Acesso em 12 mar 2026.
24. [24] *Linux man-pages — wait(2)*. Disponível em: <https://man7.org/linux/man-pages/man2/wait.2.html>. Acesso em 12 mar 2026.
25. [25] *Linux man-pages — pidfd_send_signal(2)*. Disponível em: https://man7.org/linux/man-pages/man2/pidfd_send_signal.2.html. Acesso em 12 mar 2026.
26. [26] *Linux kernel documentation — Control Group v2*. Disponível em: <https://docs.kernel.org/admin-guide/cgroup-v2.html>. Acesso em 12 mar 2026.
27. [27] *Python Documentation — C API, Operating System Utilities*. Disponível em: <https://docs.python.org/3/c-api/sys.html>. Acesso em 12 mar 2026.
28. [28] *Python Documentation — multiprocessing start methods / fork compatibility*. Disponível em: <https://docs.python.org/3/library/multiprocessing.html>. Acesso em 12 mar 2026.

29. [29] *Python Documentation — What's New in Python 3.12 (fork warning)*. Disponível em: <https://docs.python.org/3/whatsnew/3.12.html>. Acesso em 12 mar 2026.